

Pemrograman Berorientasi Objek



Introduction to OOP

Syllabus

- ▶ Introduction to OOP
- ▶ Class and Object
- ▶ Encapsulation
- ▶ Class Relationship
- ▶ Inheritance
- ▶ Abstract Class and Interface
- ▶ Polymorphism, Static, & Collection
- ▶ Exception
- ▶ UTS
- ▶ MVC Architecture
- ▶ HTML, CSS, JS
- ▶ Java Web
- ▶ JDBC
- ▶ Tubes

Grading Component

- ▶ Practicum per Week (12x) start at 2nd week (1st is runmod): 23%
- ▶ Quiz / Assignment @ LMS: 15%
- ▶ UTS: 22%
- ▶ Project Application: 30%
- ▶ Project Documentation: 5%

Course rules

- ▶ Just listen to the course 😊
- ▶ Cheating or plagiarism, max score is D
- ▶ Late? Let's define

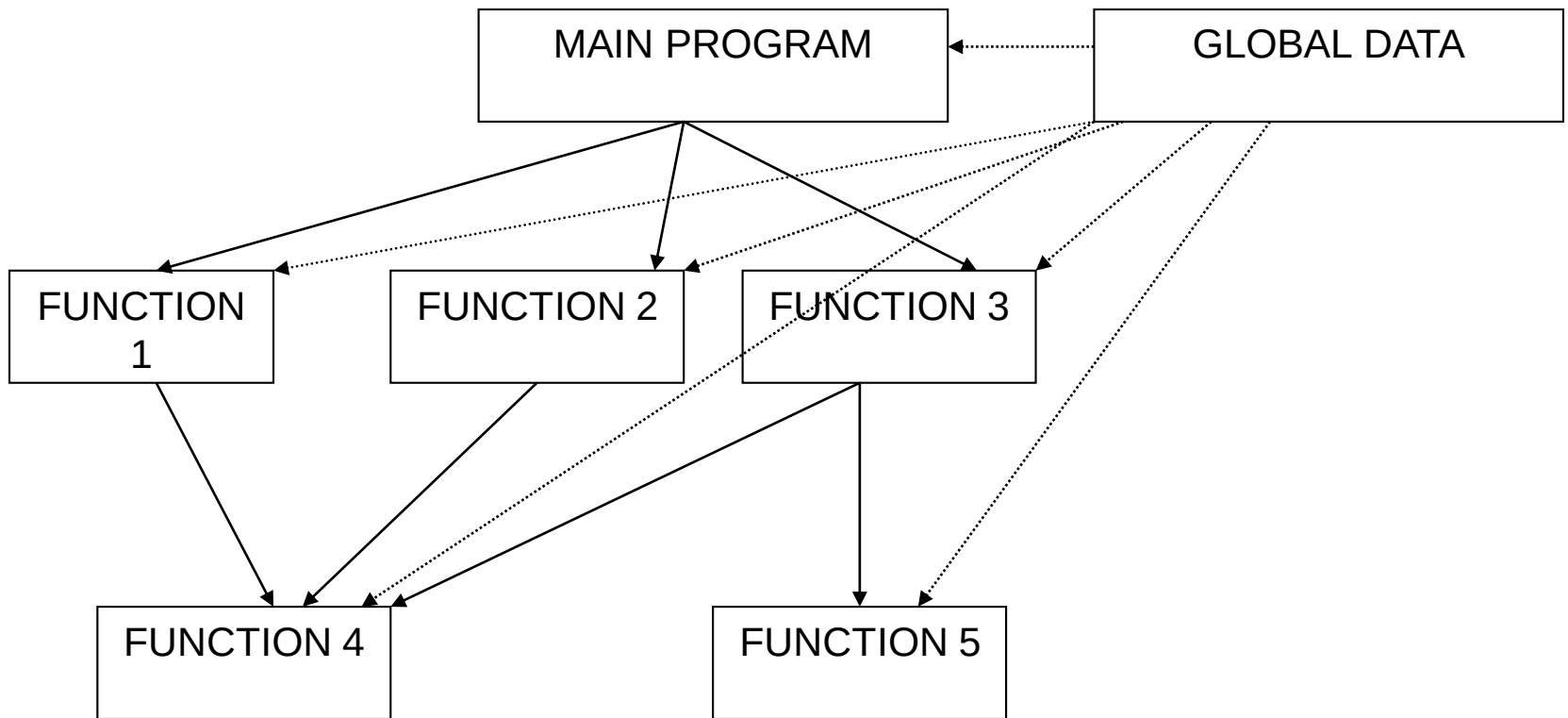
Hello Everyone!

- ...

Introduction to OOP

- ▶ Difference between structural programming and OOP
- ▶ Basic terminology in OOP
- ▶ The advantage and disadvantage of OOP
- ▶ Four design principles of OOP
- ▶ Class and Object
- ▶ Variable

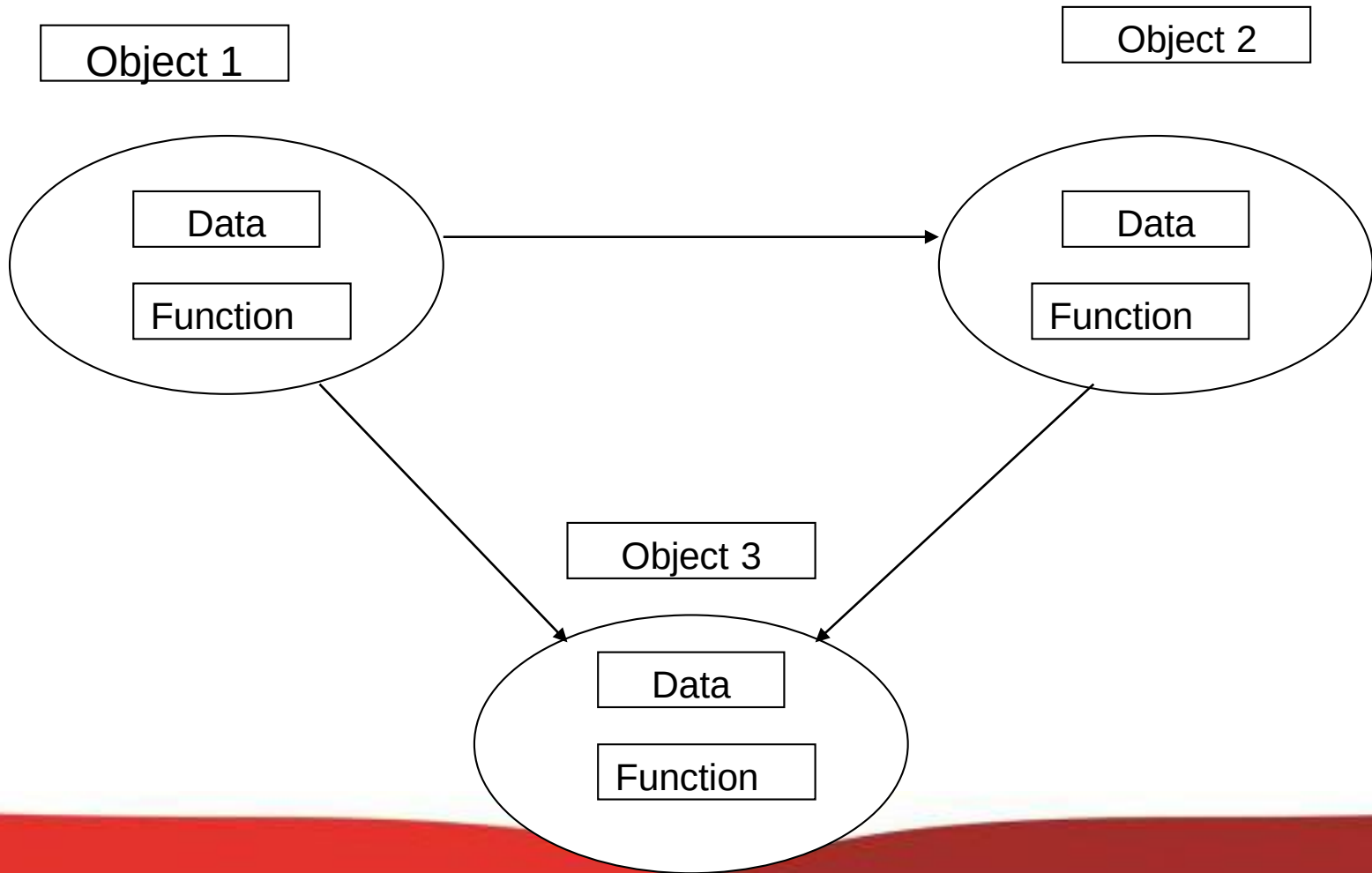
Structured Programming



Structured Programming

- ▶ Using function
- ▶ Function & program is divided into modules
- ▶ Every module has its own data and function which can be called by other modules

Object Oriented Programming



OOP

- Objects have both data and methods
- Objects of the same class have the same data elements and methods
- Objects send and receive messages to invoke actions
- The real world can be accurately described as a collection of objects that interact.

Basic Terminology in OOP

- ▶ Object: a real world entity such as pen, laptop, car, bed, keyboard, mouse, chair, transaction, user
- ▶ Class: a group or type of similar objects
- ▶ Method: what an object can do
 - A car can turn right, turn left, stop, etc
- ▶ Attribute: what an object can have
 - A student can have name, id, grade, etc

The Advantage of OOP

- **Code Reuse and Recycling**: Objects created for Object Oriented Programs can easily be reused in other programs.
- **Abstraction**: Once an Object is created, knowledge of its implementation is not necessary for its use.
- **Data Hiding**: Objects have the ability to hide certain parts of themselves from programmers. This prevents programmers from tampering with values they shouldn't.

The Advantage of OOP (Cont)

- ▶ **Design Benefits**: Object Oriented Programs force designers to go through an extensive planning phase, which makes for better designs with less flaws.
- ▶ **Software Maintenance**: An Object Oriented Program is much easier to modify and maintain than a non-Object Oriented Program.

The Disadvantage of OOP

- ▶ **Size**: Object Oriented programs are much larger than other programs.
- ▶ **Effort**: Object Oriented programs require a lot of work to create.
- ▶ **Speed**: Object Oriented programs are slower than other programs, partially because of their size. Also, OOP demand more system resources, thus slowing the program down.

Design Principle of OOP

- ▶ Abstraction
- ▶ Encapsulation
- ▶ Inheritance
- ▶ Polymorphism

Abstraction

- ▶ Focus only on the important facts about the problem at hand to design, produce, and describe so that it can be easily used without knowing the details of how it works.

Analogy:

- ▶ When you drive a car, you don't have to know how the gasoline and air are mixed and ignited.
- ▶ Instead you only have to know how to use the controls.

Encapsulation

- ▶ Also known as data hiding
- ▶ Only object's methods can modify information in the object.

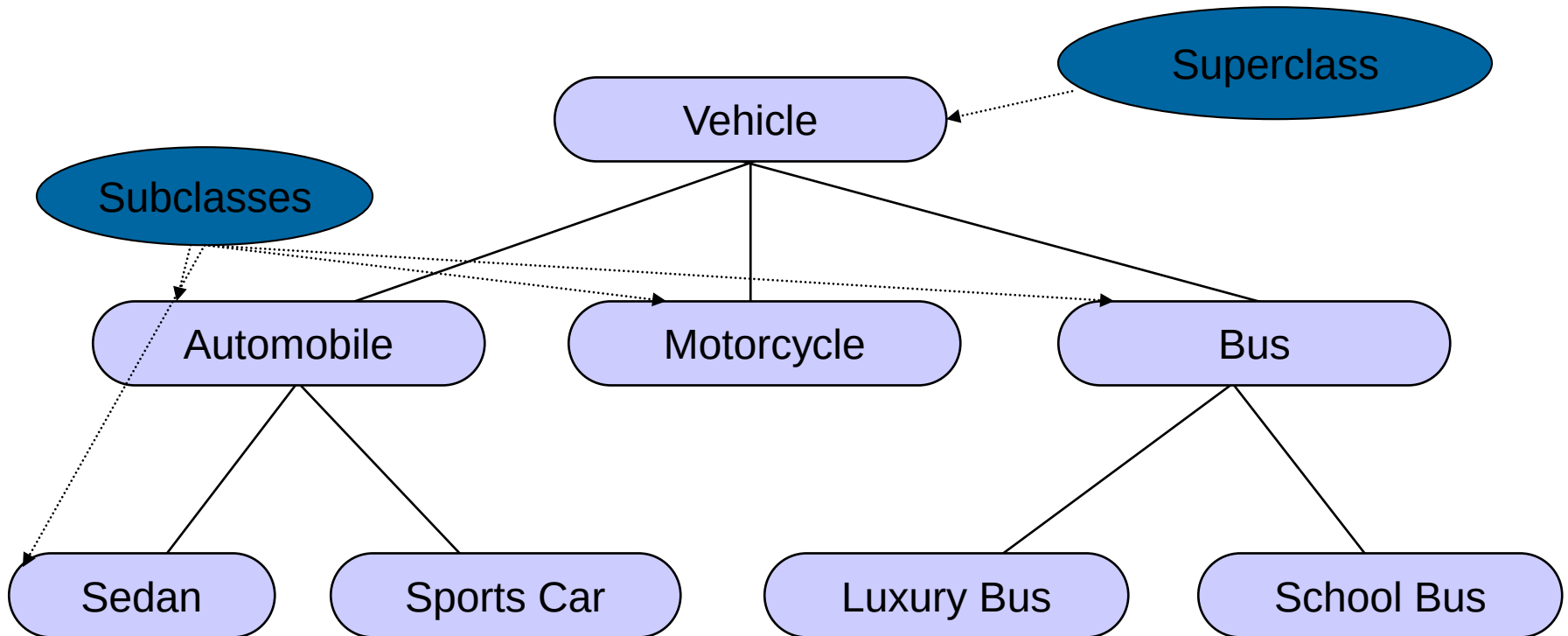
Analogy:

- ▶ ATM machine can only update accounts of one person or object only.

Inheritance

- ▶ A way of organizing classes
- ▶ Classes with properties in common can be grouped so that their common properties are only defined once.
- ▶ Superclass – inherit its attributes & methods to the subclass(es).
- ▶ Subclass – can inherit all its superclass attributes & methods besides having its own unique attributes & methods.

Define the similar and different properties!



Polymorphism

- ▶ the same object or behaviour can have more than one interpretation in different context
- ▶ Example:
 - A shape can be a triangle, a square or a circle
 - How every creature move is different. Fish move by swimming, bird move by flying, etc

Question?





CLASS

OOP

- Represent the real world: Baby

Baby

```
String name  
boolean isMale  
double weight  
double decibels  
int numPoops
```



```
// little baby alex  
String nameAlex;  
double weightAlex;  
// little baby david  
String nameDavid;  
double weightDavid;  
// little baby david  
String nameDavid2;  
double weightDavid2;
```



David2?
Terrible 😞

500 Babies? That Sucks!

Cont.



Baby1



Baby2



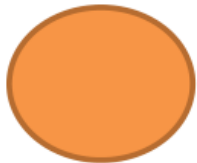
Baby3



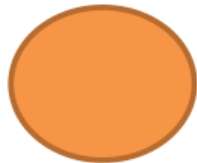
Baby4

496
more
Babies
...

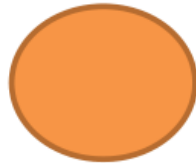
Cont.



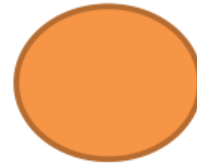
Nurse1



Nurse2



Nurse3



Nurse4

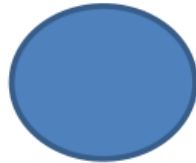
More nurses...



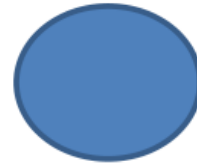
Baby1



Baby2



Baby3



Baby4

496 more
Babies ...



Creating Class

```
public class Baby {
```

fields

methods

```
}
```

Notes

- ▶ Class name is capitalize
- ▶ 1 class = 1 file
- ▶ Having main methods means the class can be run

Writing class

```
public class Baby {  
    String name;  
    boolean isMale;  
    double weight;  
    double decibels;  
    int numPoops = 0;  
  
    void poop() {  
        numPoops += 1;  
        System.out.println("Dear mother, "+  
            "I have pooped.  Ready the diaper.");  
    }  
}
```

Design Class

Start from concrete object then generalize

- Things that object **know**
- Things that object **does**



ShoppingCart
cartContents
addToCart() removeFromCart() checkout()

knows

does

Alarm
alarmTime alarmMode
setAlarmTime() getAlarmTime() isAlarmSet() snooze()

knows

does

Button
label color
setColor() setLabel() dePress() unDepress()

knows

does

Cont.

**instance
variables**

(state)

methods

(behavior)

Song
title artist
setTitle() setArtist() play()

knows

does

Television



**instance
variables**

methods



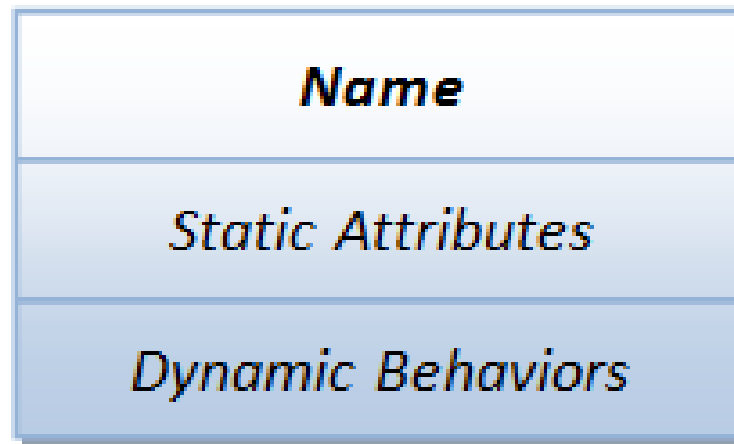
Cont.

Name (Identifier) Variables (Static attributes) Methods (Dynamic behaviors)	Student	Circle
	name grade	radius color
	getName() printGrade()	getRadius() getArea()
	SoccerPlayer	Car
	name number xLocation yLocation	plateNumber xLocation yLocation speed
	run() jump() kickBall()	move() park() accelerate()

Examples of classes

Summary

- ▶ *a class is a definition of objects of the same kind*
- ▶ *Name* (or identity): identifies the class.
- ▶ *Variables* (or attribute, state, field): contains the *static attributes* of the class.
- ▶ *Methods* (or behaviors, function, operation): contains the *dynamic behaviors* of the class.



A class is a 3-compartment box

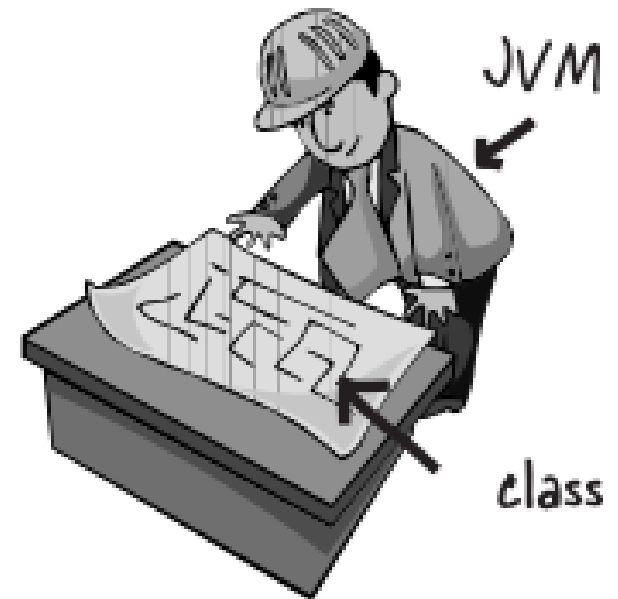
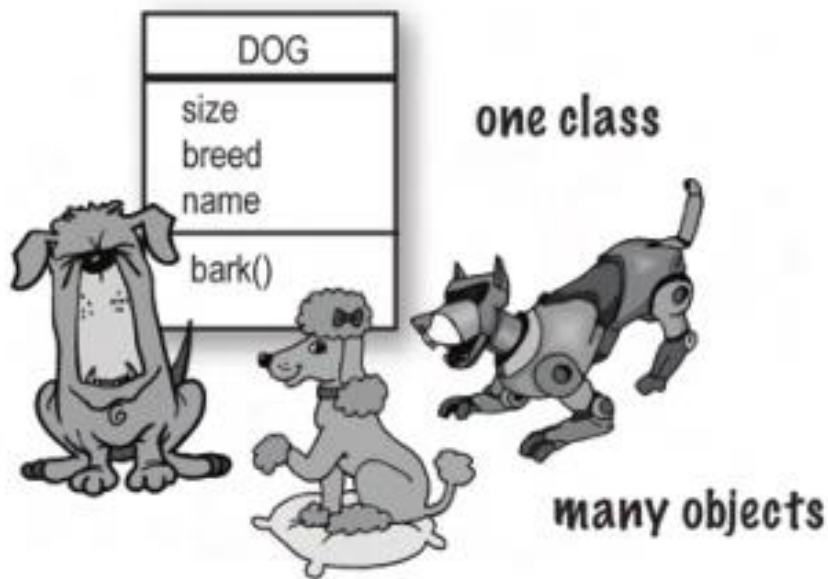


Fakultas Informatika
School of Computing
Telkom University

OBJECT

Class is not object

- ▶ One class, many objects
- ▶ Class used to construct objects (using new keyword)
- ▶ Class is “blueprint” or “recipe” for object



Making your object run

1. Write your class
2. Write tester (driver) class
3. In your tester, make an object and access the object's variable and methods

Write class

```
class Dog{
```

```
    int size;
```

```
    String breed;
```

```
    String name;
```

```
    void bark(){
```

```
        System.out.println("Wof! Wof!");
```

```
    }
```

```
}
```



Write tester/driver class

```
class DogTest {  
    public static void main (String[] args){  
        // test will be in here  
    }  
}
```

Write test

```
class DogTest {  
    public static void main (String[] args){  
        Dog d = new Dog(); // make object (term: instantiate)  
        d.size = 40; // set size of dog  
        d.bark(); //call bark methods  
    }  
}
```


Two use of main (tester/driver class)

- ▶ Test your real test
 - It is possible to have main in each class to test class itself
- ▶ To launch/start your java Apps
 - main is entry point

OOP Characteristics

1. Everything is an object
2. A program is bunch of objects telling each other what to do by sending messages
3. Each object has its own memory made up of other object
4. Every object has a type
5. All object of particular type can receive the same message

More compact definition

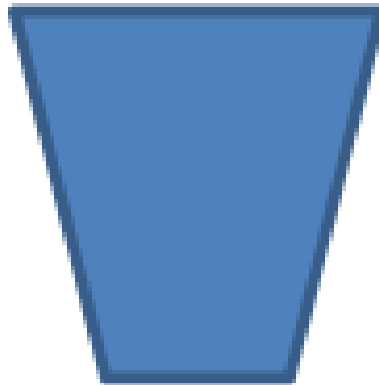
An Object has state, behavior and identity



VARIABLE

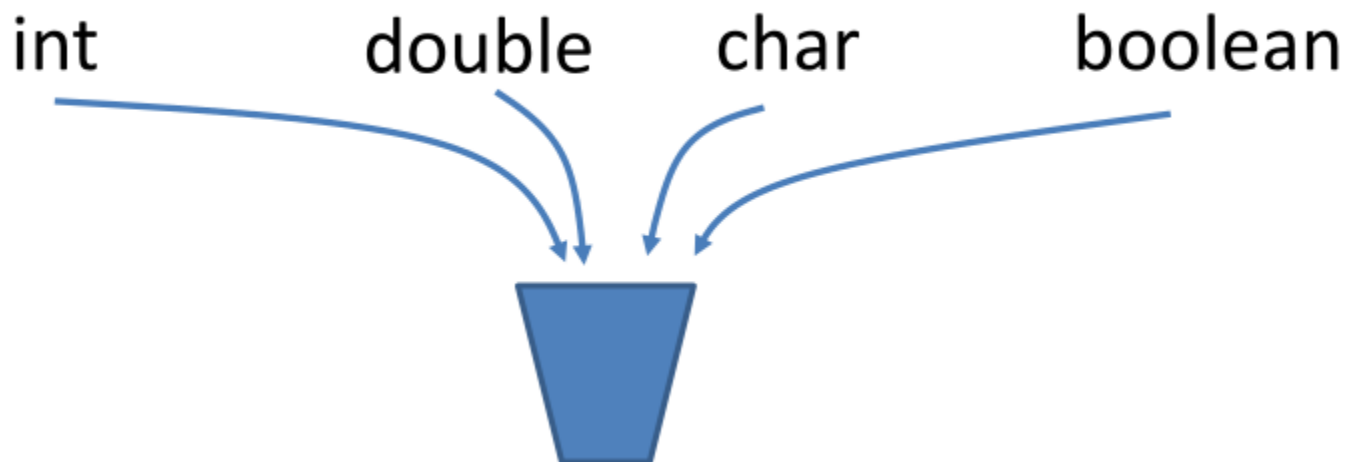
What variable does?

- ▶ Variable is just a cup
- ▶ A container. It hold something



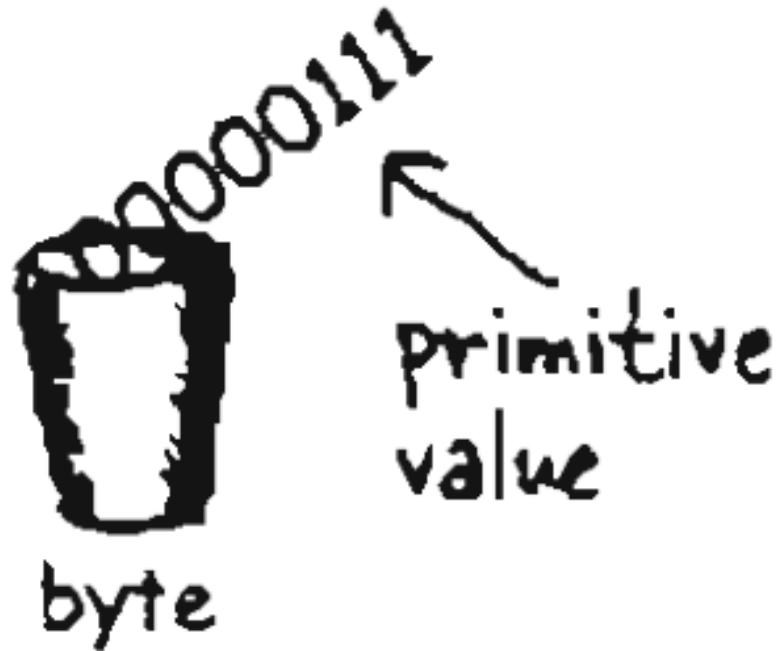
Primitive type variable

- ▶ int, long, double, boolean, char, short, byte, float
- ▶ The actual values are stored in the variable
- ▶ Primitives are small enough that they just fit into the cup



Example

byte x = 7;



Reference type variable

- ▶ Everything except primitives type: object, array
- ▶ Objects are too big to fit in a variable (cup)



Cont.

- There is actually no such thing as an object variable
- There's only an **object reference** variable
- An object reference variable holds bits that represent a way to access an object
- It doesn't hold the object itself, but it holds something like a pointer (but not pointer like C)

You manipulate object with reference

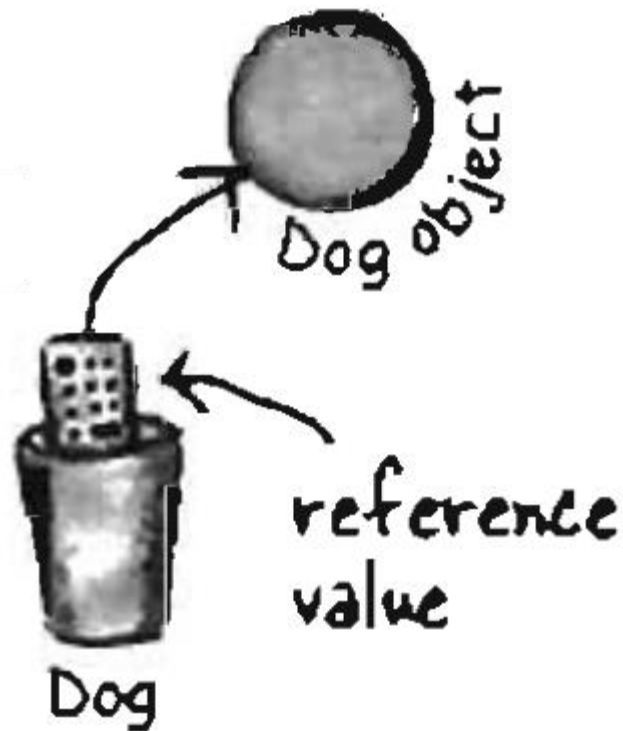
- ▶ Although you treat everything as an object, the identifier you manipulate is actually a “reference” to an object.
- ▶ You might imagine a television (the object) and a remote control (the reference).
- ▶ As long as you’re holding this reference, you have a connection to the television, but when someone says, “Change the channel” or “Lower the volume,” what you’re manipulating is the reference, which in turn modifies the object.
- ▶ If you want to move around the room and still control the television, you take the remote/reference with you, not the television.

Reference as remote



Example

```
Dog myDog = new Dog();
```



3 Steps Object creation

1. Declare reference variable

- Dog myDog;

2. Create an object

- new Dog()

3. Link object to reference

- Dog myDog = new Dog();

Where program lives

1. Register (inside processor)
2. The stack (in RAM but have direct support from processor)
3. The heap (also in RAM, very flexible)
4. Constant storage (ROM)

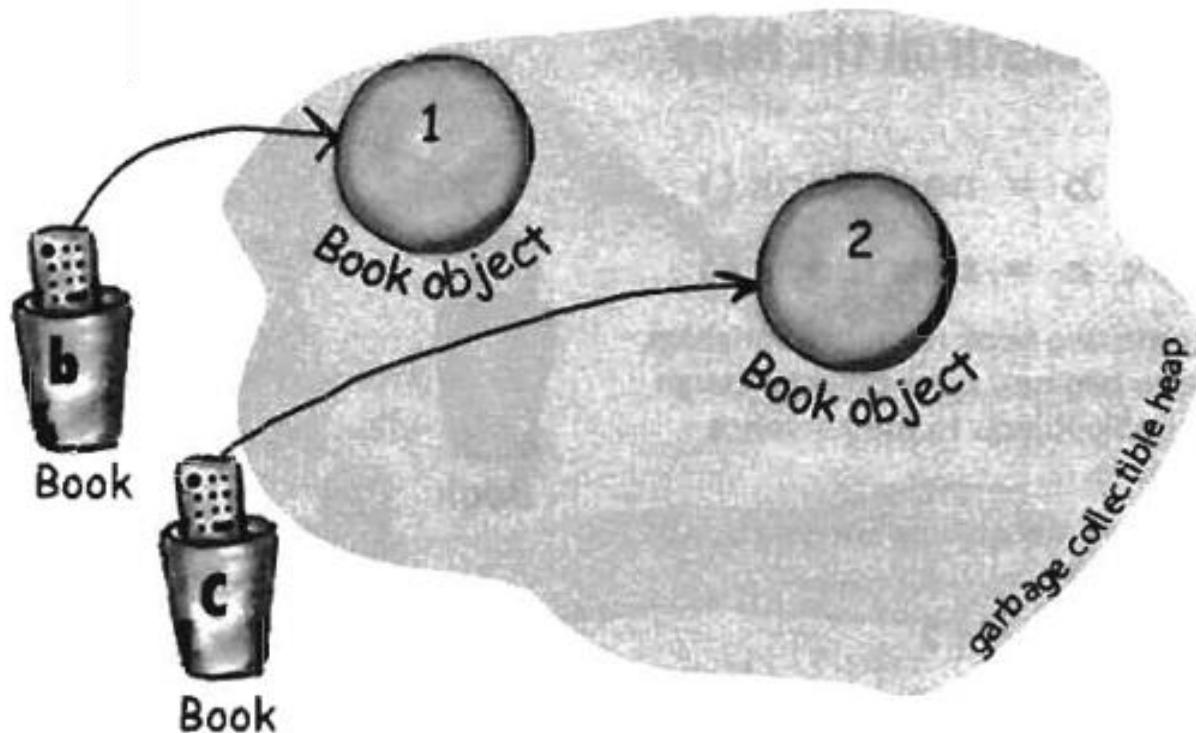
3 things to remember

- **Instance variable** and **object** live on **heap**
- **Local variable** live on **stack**

Life and death of object

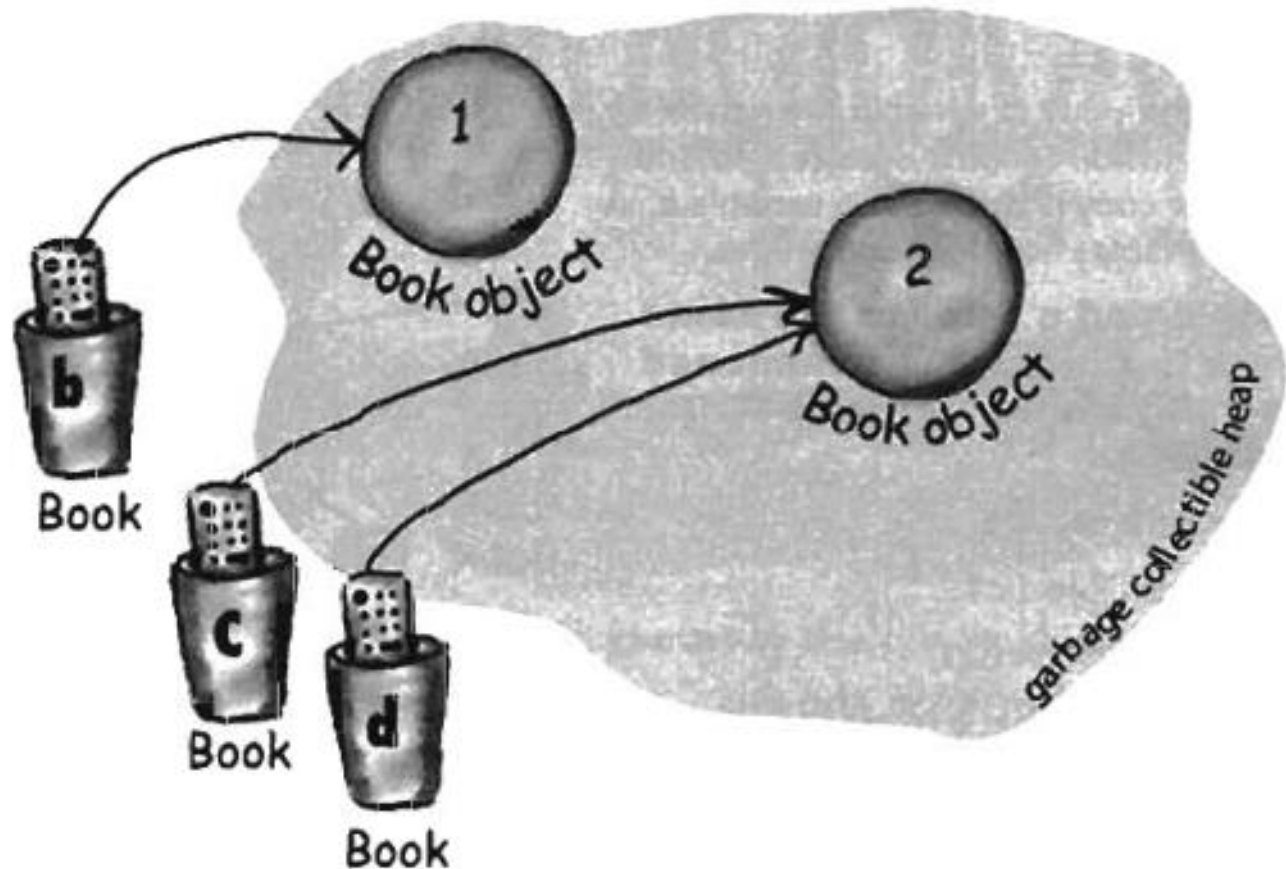
Book b = new Book();

Book c = new Book();



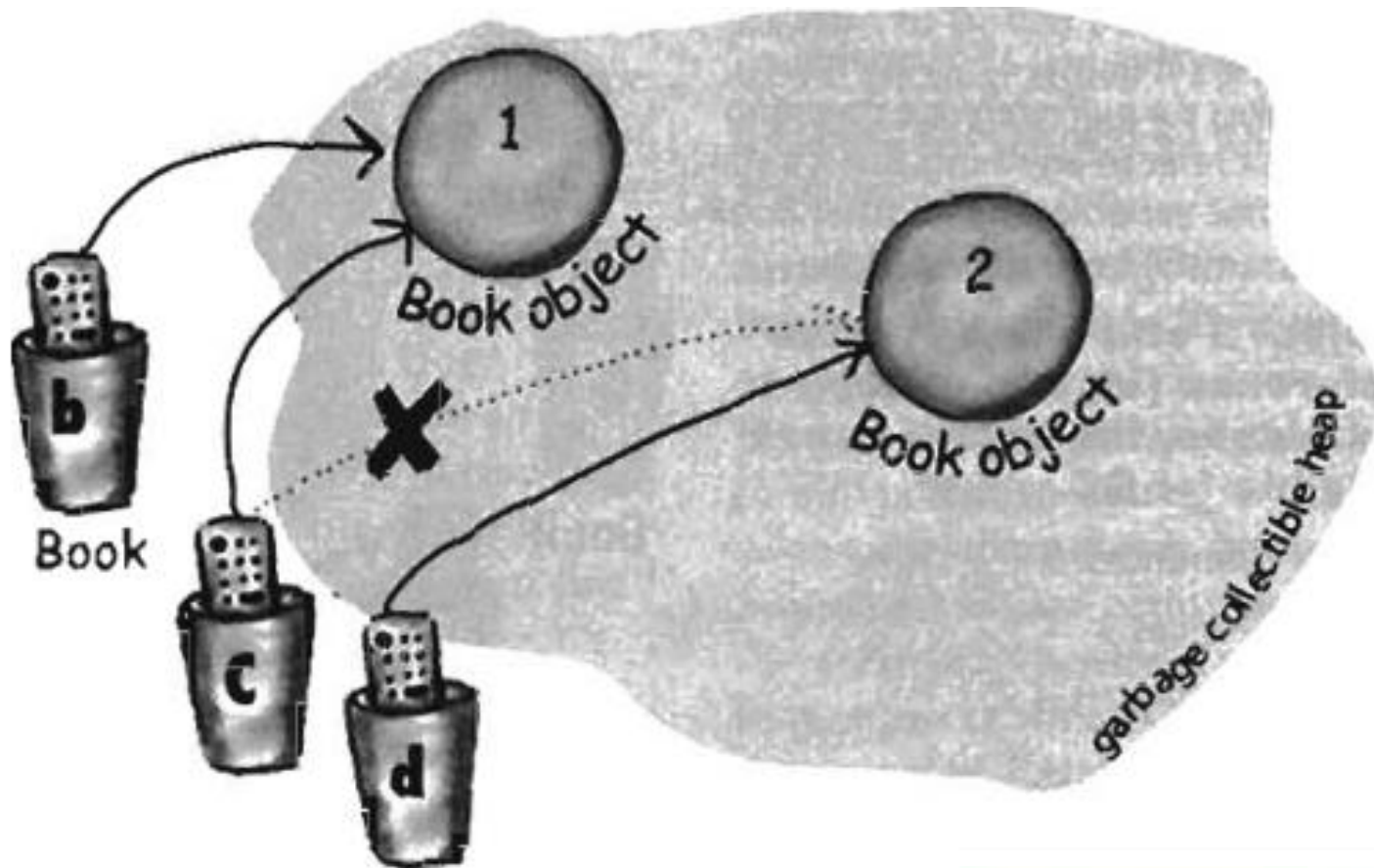
Cont.

Book d = c;



Cont.

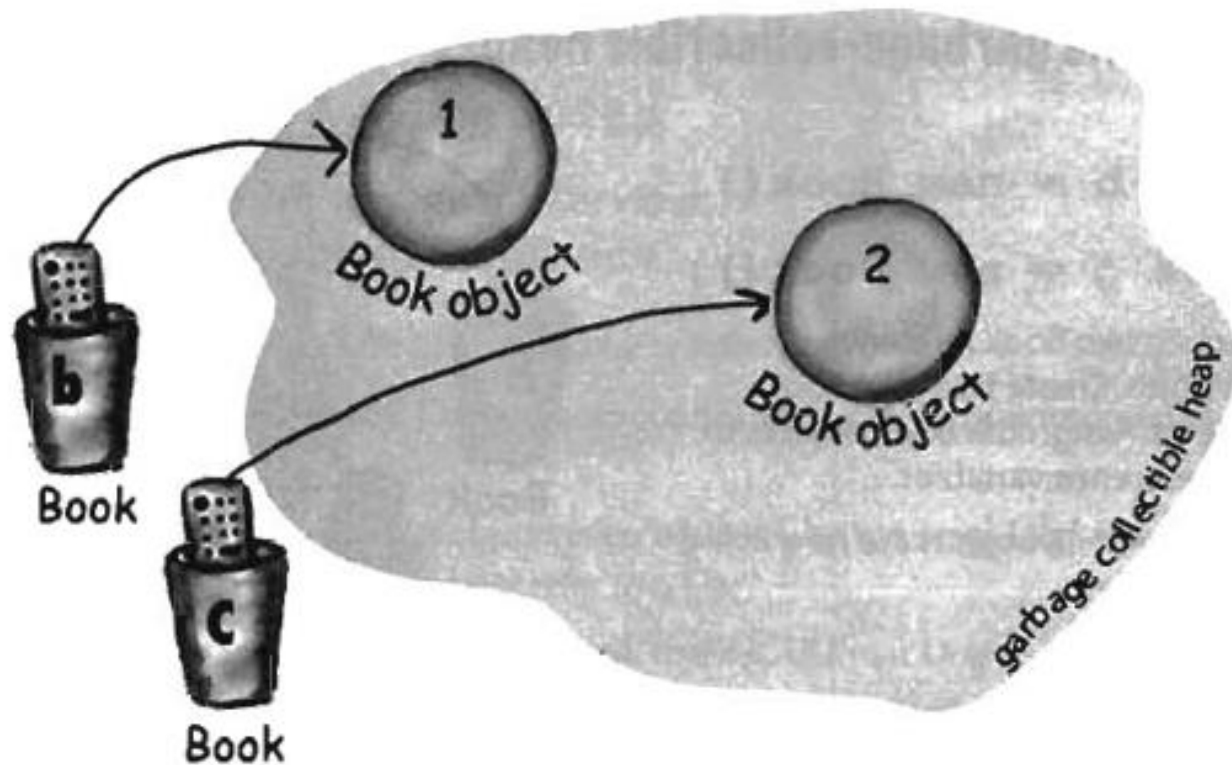
`c = b;`



Cont.

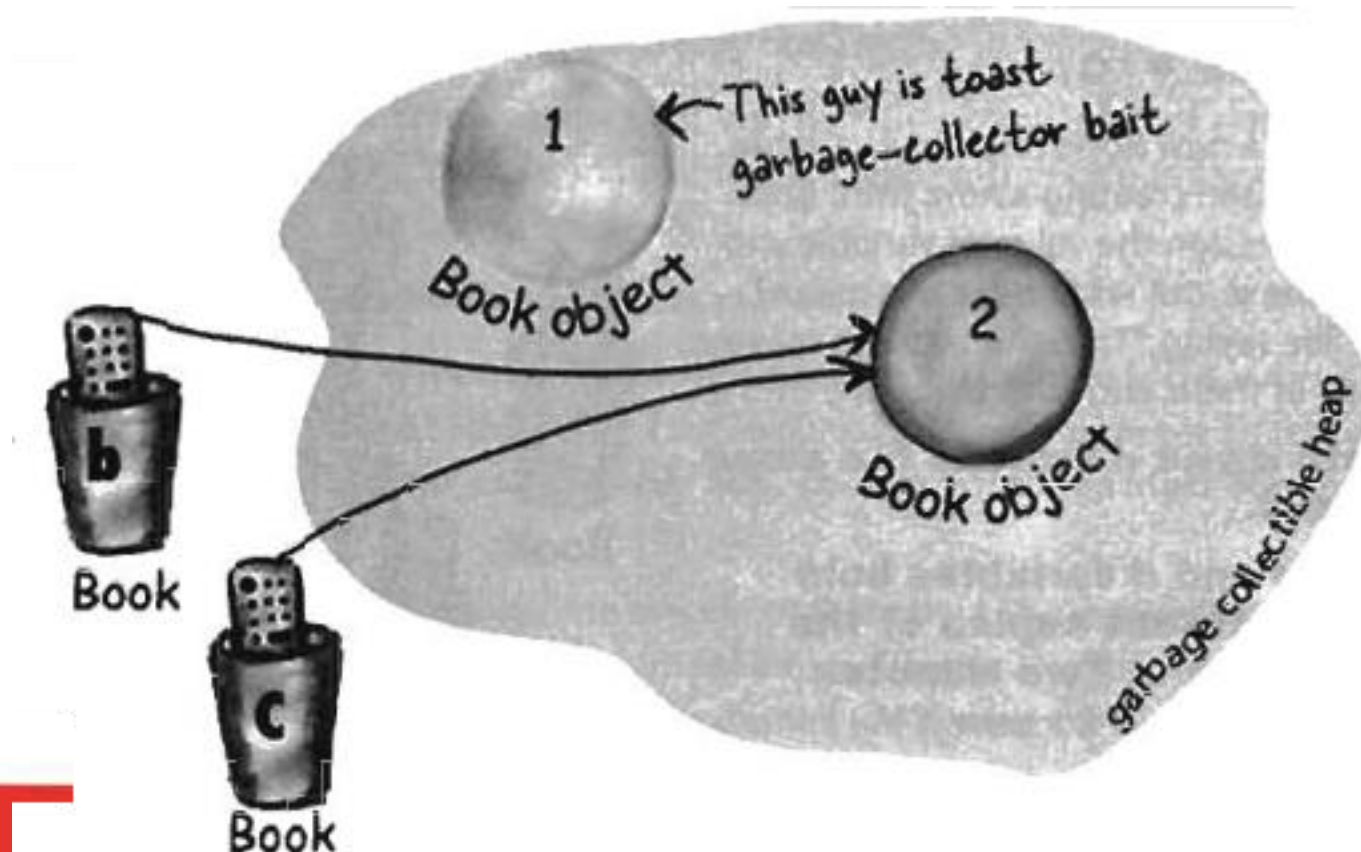
Book b = new Book();

Book c = new Book();



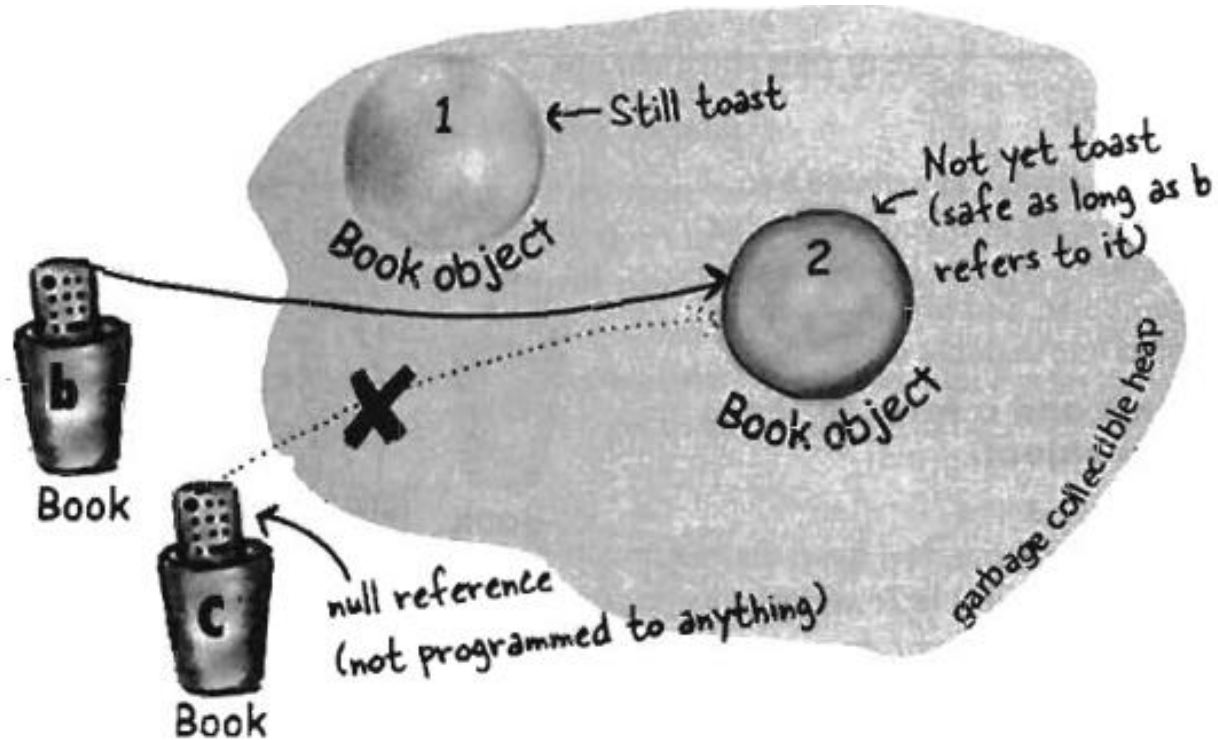
Cont.

`b = c;`



Cont.

`c = null;`

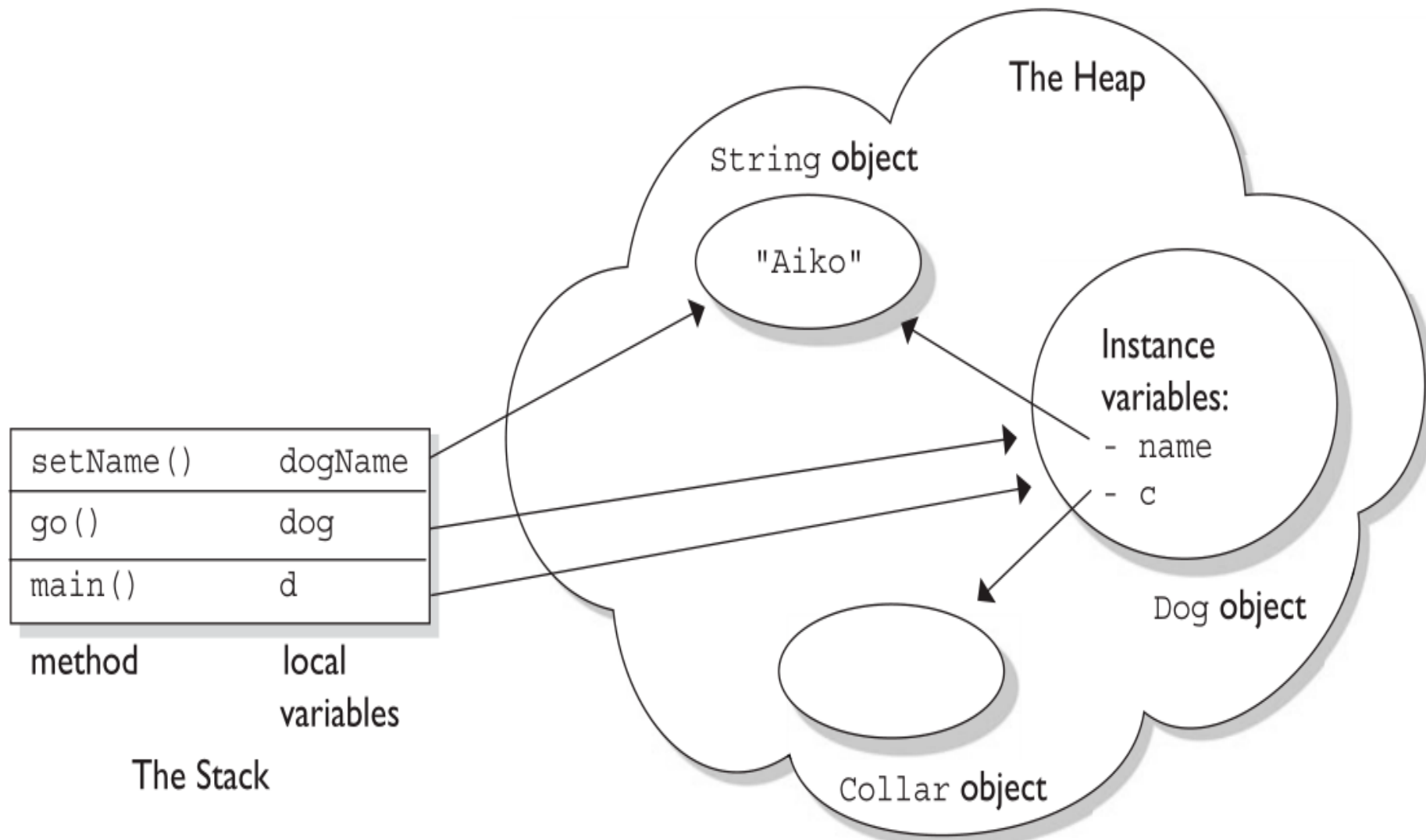




Complex example

```
1. class Collar { }
2.
3. class Dog {
4.     Collar c;           // instance variable
5.     String name;        // instance variable
6.
7.     public static void main(String [] args) {
8.
9.         Dog d;           // local variable: d
10.        d = new Dog();
11.        d.go(d);
12.    }
13.    void go(Dog dog) {    // local variable: dog
14.        c = new Collar();
15.        dog.setName("Aiko");
16.    }
17.    void setName(String dogName) { // local var: dogName
18.        name = dogName;
19.        // do more stuff
20.    }
21. }
```

Ilustration (create animation)

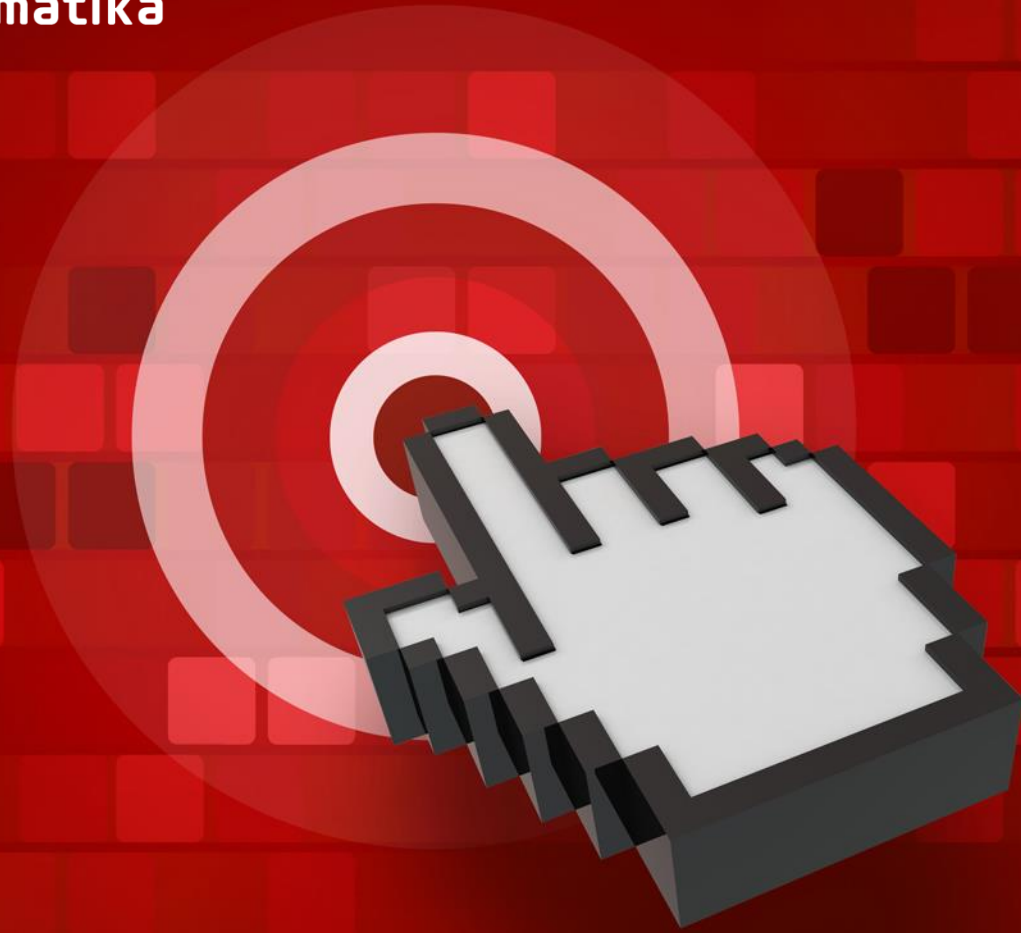


Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU